PASSWORD STRENGTH ANALYZER

Introduction to Analysis of Algorithms

B.Tech Computer Science and Engineering
Implementation Language: Python 3

Project objective: Create a local educational program that analyzes basic password characteristics, gives a heuristic strength score, and demonstrates algorithm design and complexity analysis.

1. Abstract

This project builds a Password Strength Analyzer in Python. It evaluates password length, character classes, a small common-password list, and simple repeated-character or sequence patterns. It then calculates a transparent score, gives a descriptive rating, and suggests improvements. A rough entropy estimate is included to demonstrate logarithms and search-space reasoning. The project is educational: its score is not a guarantee of real-world security.

2. Problem Statement

Short and predictable passwords are easier to guess. The project aims to identify basic weaknesses in a sample password and provide understandable feedback while applying algorithm design, string processing, testing, and Big-O analysis.

3. Objectives

• Check password length and character variety.

• Detect a small list of common passwords and simple predictable patterns.

• Generate a score and improvement suggestions.

• Estimate theoretical entropy from length and character pool.

• Measure runtime for inputs of increasing length and analyze complexity.

4. Scope and Assumptions

The program runs locally and does not save or transmit the input. It uses a small built-in common-password list and fixed pattern checks. It does not query breached-password databases. Entropy assumes random independent character choices, which is often not true for human-created passwords.

5. Algorithm Workflow

• Read a sample password.

• Scan for length, lowercase, uppercase, digits, symbols, and whitespace.

• Check the common-password list and simple repeat/sequence patterns.

• Add points for criteria that pass, then cap the score at 100.

• Map the score to a rating and generate suggestions for failed checks.

• Estimate character pool size and rough entropy, then display the result.

6. Pseudocode

```
INPUT password
checks = evaluate length, character classes, common list, and patterns
score = 0
FOR each scoring rule:
    IF rule passes:
        score = score + rule_weight
score = MIN(score, 100)
rating = map score to a strength label
pool = estimate character pool size
IF pool > 0:
    entropy = length(password) * log2(pool)
ELSE:
    entropy = 0
feedback = suggestions for failed checks
OUTPUT score, rating, checks, entropy, feedback
```

7. Implementation Details

The main function is analyze_password() in password_analyzer/app.py. estimate_character_pool() determines which character classes occur and estimates a possible pool size. Python standard-library modules are used, so no external packages are required. benchmark.py uses timeit and synthetic sample strings to measure runtime at several input lengths.

8. Complexity Analysis

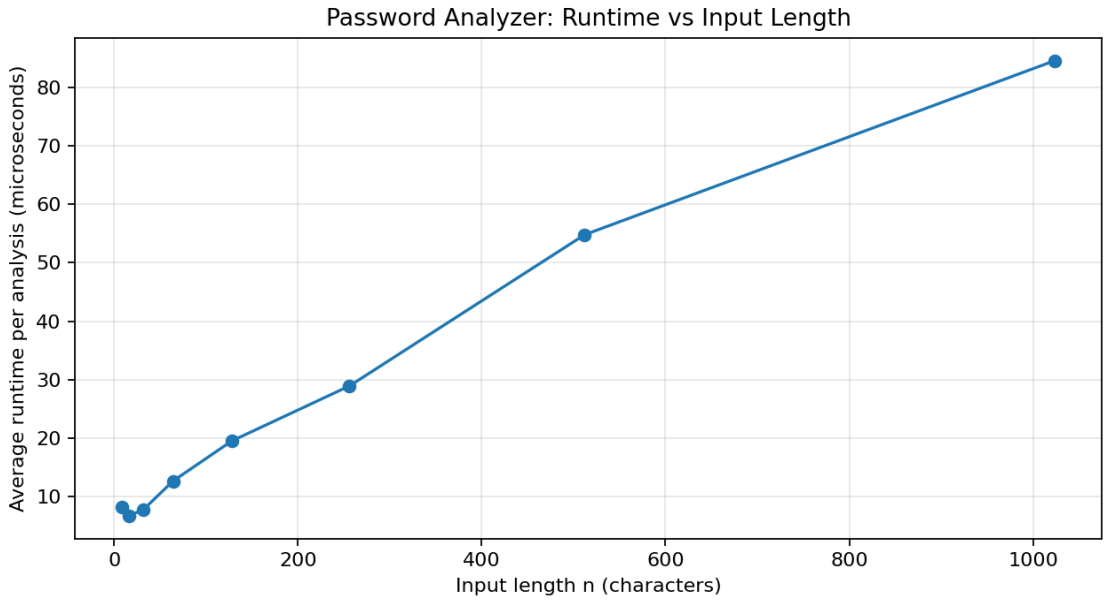
Operation	Complexity	Explanation
Character-class and length checks	O(n)	A fixed number of scans over n characters.
Common-password set lookup	O(1) average	The set is fixed-size and uses hash lookup.
Scoring and rating mapping	O(1)	Fixed number of rules after scans.
Overall time	O(n)	A constant number of linear scans dominates.
Auxiliary space	O(1)	Fixed checks and feedback structure, excluding input/output.

Note: the regex patterns are fixed and simple; regex worst-case behavior depends on pattern structure. The input length n is the main size parameter.

9. Runtime Experiment

benchmark.py measures repeated analyses of synthetic strings with lengths 8, 16, 32, 64, 128, 256, 512, and 1024. It writes benchmark_results.csv and the included graph visualizes this run. Very short-run measurements vary with system load and hardware, so use the graph as an experiment rather than proof of complexity. The theoretical analysis remains O(n).

Runtime graph from this build:



10. Testing

Four unit tests are included: empty input behavior, common-password detection, relative score for short versus mixed long sample, and character-pool calculation. Run them using: python3 -m unittest discover -s tests -v. All four tests passed during this build.

11. Five-Member Work Division

• Member 1: Problem statement, objectives, algorithm design, pseudocode, and flowchart.

• Member 2: Python implementation, scoring rules, pattern checks, and code integration.

• Member 3: Big-O analysis, runtime benchmarking, results table, and graph.

• Member 4: Unit tests, sample inputs, expected/actual results, and bug fixes.

• Member 5: Report and presentation compilation, demo coordination, conclusion, and future scope.

All members should understand the complete program. Member 5 should compile the report, but each member should write or contribute their assigned section.

12. Limitations and Ethical Considerations

• The score is a heuristic, not a standardized security rating.

• The common-password list is small and cannot identify every weak password.

• Entropy is a rough theoretical estimate and assumes randomness.

• The tool should not be used to collect or store real passwords.

• Use unique passwords, a reputable password manager, and multi-factor authentication where available.

13. Conclusion and Future Work

The project demonstrates algorithm design, string processing, decision rules, testing, and empirical performance measurement. Its main algorithm has O(n) time complexity and O(1) auxiliary space under the stated assumptions. Future improvements could include a larger offline dictionary, stronger pattern detection, a graphical interface, and a broader test suite.

14. How to Run

From the extracted project folder, run `python3 -m password_analyzer.app`. To test, run `python3 -m unittest discover -s tests -v`. To benchmark, run `python3 benchmark.py`. Use only demonstration strings in class.